



CUDA Zero Copy Mapped Memory

📅 12-16-2022 📅 12-16-2022 📁 BLOG ⌚ 10 MINUTES READ (ABOUT 1564 WORDS) 🔒

Introduction

Unified memory is used on NVIDIA embedding platforms, such as NVIDIA Drive series and NVIDIA Jetson series. Since the same memory is used for both the CPU and the integrated GPU, it is possible to eliminate the CUDA memory copy between host and device that normally happens on a system that uses discrete GPU so that the GPU can directly access the outputs from CPU and the CPU can also directly access the outputs from GPU. In this way, the system performance could be improved significantly in some use cases.

In this blog post, I would like to discuss the CUDA mapped pinned memory versus CUDA non-mapped pinned memory and compare their performance on memory bound kernels.

CUDA Pinned Mapped Memory

CUDA pinned mapped memory enables GPU threads to directly access host memory. For this purpose, it requires mapped [pinned \(non-pageable, page-locked\) memory](#). On integrated GPUs (i.e., GPUs with the integrated field of the CUDA device properties structure set to 1), mapped pinned memory is always a performance gain because it avoids superfluous copies as integrated GPU and CPU memory are physically the same. On discrete GPUs, mapped pinned memory is advantageous only in certain cases. Because the data is not cached on the GPU, mapped pinned memory should be read or written only once, and the global loads and stores that read and write the memory should be coalesced. Zero copy can be used in place of streams because kernel-originated data transfers automatically overlap kernel execution without the overhead of setting up and determining the optimal number of streams.

CUDA Pinned Memory Non-Mapped VS Mapped

The following implementation compares the latency of a memory-bound kernel and its memory copy between host and device if necessary.

CUDA mapped memory also uses pinned memory. For CUDA pinned memory, we still need to allocate device memory and transfer the data between the host memory and the device memory, whereas for CUDA

mapped memory, the device memory allocation and memory transfer, if there is any, are abstracted.

▼ mapped_memory.cu



```

1  #include <cassert>
2  #include <chrono>
3  #include <functional>
4  #include <iomanip>
5  #include <iostream>
6  #include <stdexcept>
7  #include <thread>
8  #include <tuple>
9  #include <utility>
10 #include <vector>
11
12 #include <cuda_runtime.h>
13
14 #define CHECK_CUDA_ERROR(val) check((val), #val, __FILE_
15 void check(cudaError_t err, const char* const func, con:
16           const int line)
17 {
18     if (err != cudaSuccess)
19     {
20         std::cerr << "CUDA Runtime Error at: " << file <
21                 << std::endl;
22         std::cerr << cudaGetErrorString(err) << " " << :
23         std::exit(EXIT_FAILURE);
24     }
25 }
26
27 #define CHECK_LAST_CUDA_ERROR() checkLast(__FILE__, __L:
28 void checkLast(const char* const file, const int line)
29 {
30     cudaError_t const err(cudaGetLastError());
31     if (err != cudaSuccess)
32     {
33         std::cerr << "CUDA Runtime Error at: " << file <
34                 << std::endl;
35         std::cerr << cudaGetErrorString(err) << std::en:
36         std::exit(EXIT_FAILURE);
37     }
38 }
39
40 template <class T>
41 float measure_performance(std::function<T(cudaStream_t)>
42                           cudaStream_t stream, int num_,
43                           int num_warmups = 100)
44 {
45     cudaEvent_t start, stop;
46     float time;
47
48     CHECK_CUDA_ERROR(cudaEventCreate(&start));
49     CHECK_CUDA_ERROR(cudaEventCreate(&stop));
50
51     for (int i{0}; i < num_warmups; ++i)
52     {
53         bound_function(stream);
54     }

```

```

55
56     CHECK_CUDA_ERROR(cudaStreamSynchronize(stream));
57
58     CHECK_CUDA_ERROR(cudaEventRecord(start, stream));
59     for (int i{0}; i < num_repeats; ++i)
60     {
61         bound_function(stream);
62     }
63     CHECK_CUDA_ERROR(cudaEventRecord(stop, stream));
64     CHECK_CUDA_ERROR(cudaEventSynchronize(stop));
65     CHECK_LAST_CUDA_ERROR();
66     CHECK_CUDA_ERROR(cudaEventElapsedTime(&time, start,
67     CHECK_CUDA_ERROR(cudaEventDestroy(start));
68     CHECK_CUDA_ERROR(cudaEventDestroy(stop));
69
70     float const latency{time / num_repeats};
71
72     return latency;
73 }
74
75 __global__ void float_addition(float* output, float const
76     float const* input_2, ui
77 {
78     const uint32_t idx{blockDim.x * blockIdx.x + thread
79     const uint32_t stride{blockDim.x * gridDim.x};
80     for (uint32_t i{idx}; i < n; i += stride)
81     {
82         output[i] = input_1[i] + input_2[i];
83     }
84 }
85
86 void launch_float_addition_non_mapped_pinned_memory(
87     float* h_output, float const* h_input_1, float const
88     float* d_output, float* d_input_1, float* d_input_2,
89     cudaStream_t stream)
90 {
91     CHECK_CUDA_ERROR(cudaMemcpyAsync(d_input_1, h_input_
92         cudaMemcpyHostToDevice);
93     CHECK_CUDA_ERROR(cudaMemcpyAsync(d_input_2, h_input_
94         cudaMemcpyHostToDevice);
95     dim3 const threads_per_block{1024};
96     dim3 const blocks_per_grid{32};
97     float_addition<<<blocks_per_grid, threads_per_block,
98         d_output, d_input_1, d_input_2, n);
99     CHECK_LAST_CUDA_ERROR();
100     CHECK_CUDA_ERROR(cudaMemcpyAsync(h_output, d_output,
101         cudaMemcpyDeviceToHost);
102 }
103
104 void launch_float_addition_mapped_pinned_memory(float* c
105     float* c
106     float* c
107     cudaStre
108 {
109     dim3 const threads_per_block{1024};
110     dim3 const blocks_per_grid{32};
111     float_addition<<<blocks_per_grid, threads_per_block,
112         d_output, d_input_1, d_input_2, n);
113     CHECK_LAST_CUDA_ERROR();

```



Lei Mao

*Artificial Intelligence
Machine Learning
Computer Science*

📍 Santa Clara, California

POSTS CATEGORIES TAGS

873 **7** **569**

🔄 Follow

❤️ Sponsor



CUDA Zero Copy Mapped Memory - Lei Mao's Log Book

```

114     }
115
116     void initialize_host_memory(float* h_buffer, uint32_t n,
117     {
118         for (int i{0}; i < n; ++i)
119         {
120             h_buffer[i] = value;
121         }
122     }
123
124     bool verify_host_memory(float* h_buffer, uint32_t n, float
125     {
126         for (int i{0}; i < n; ++i)
127         {
128             if (h_buffer[i] != value)
129             {
130                 return false;
131             }
132         }
133         return true;
134     }
135
136     int main()
137     {
138         constexpr int const num_repeats{10};
139         constexpr int const num_warmups{10};
140
141         constexpr int const n{1000000};
142         cudaStream_t stream;
143         CHECK_CUDA_ERROR(cudaStreamCreate(&stream));
144
145         float const v_input_1{1.0f};
146         float const v_input_2{1.0f};
147         float const v_output{0.0f};
148         float const v_output_reference{v_input_1 + v_input_2};
149
150         cudaDeviceProp prop;
151         CHECK_CUDA_ERROR(cudaGetDeviceProperties(&prop, 0));
152         if (!prop.canMapHostMemory)
153         {
154             throw std::runtime_error{"Device does not support
155         }
156
157         float *h_input_1, *h_input_2, *h_output;
158         float *d_input_1, *d_input_2, *d_output;
159
160         float *a_input_1, *a_input_2, *a_output;
161         float *m_input_1, *m_input_2, *m_output;
162
163         CHECK_CUDA_ERROR(cudaMallocHost(&h_input_1, n * sizeof(float));
164         CHECK_CUDA_ERROR(cudaMallocHost(&h_input_2, n * sizeof(float));
165         CHECK_CUDA_ERROR(cudaMallocHost(&h_output, n * sizeof(float));
166
167         CHECK_CUDA_ERROR(cudaMalloc(&d_input_1, n * sizeof(float));
168         CHECK_CUDA_ERROR(cudaMalloc(&d_input_2, n * sizeof(float));
169         CHECK_CUDA_ERROR(cudaMalloc(&d_output, n * sizeof(float));
170
171         CHECK_CUDA_ERROR(
172             cudaHostAlloc(&a_input_1, n * sizeof(float), cuda

```



Build your website for just
\$3.88/mth. More value and
performance with
Namecheap.

ADS VIA CARBON

CATALOGUE

- 1 Introduction
- 2 CUDA Pinned Mapped Memory
- 3 **CUDA Pinned Memory Non-Mapped VS Mapped**
 - 3.1 Discrete GPU
 - 3.2 Integrated GPU
- 4 Caveats
- 5 References

CUDA Zero Copy Mapped Memory - Lei Mao's Log Book

```

173 CHECK_CUDA_ERROR(
174     cudaHostAlloc(&a_input_2, n * sizeof(float), cu
175 CHECK_CUDA_ERROR(
176     cudaHostAlloc(&a_output, n * sizeof(float), cud
177
178 CHECK_CUDA_ERROR(cudaHostGetDevicePointer(&m_input_1,
179 CHECK_CUDA_ERROR(cudaHostGetDevicePointer(&m_input_2,
180 CHECK_CUDA_ERROR(cudaHostGetDevicePointer(&m_output,
181
182 // Verify the implementation correctness.
183 initialize_host_memory(h_input_1, n, v_input_1);
184 initialize_host_memory(h_input_2, n, v_input_2);
185 initialize_host_memory(h_output, n, v_output);
186 launch_float_addition_non_mapped_pinned_memory(
187     h_output, h_input_1, h_input_2, d_output, d_inpu
188     stream);
189 CHECK_CUDA_ERROR(cudaStreamSynchronize(stream));
190 assert(verify_host_memory(h_output, n, v_output_refe
191
192 initialize_host_memory(a_input_1, n, v_input_1);
193 initialize_host_memory(a_input_2, n, v_input_2);
194 initialize_host_memory(a_output, n, v_output);
195 launch_float_addition_mapped_pinned_memory(m_output,
196     n, stream
197
198 CHECK_CUDA_ERROR(cudaStreamSynchronize(stream));
199 assert(verify_host_memory(a_output, n, v_output_refe
200
201 // Measure latencies.
202 std::function<void(cudaStream_t)> function_non_mappe
203     std::bind(launch_float_addition_non_mapped_pinn
204         h_input_1, h_input_2, d_output, d_inpu
205         std::placeholders::_1));
206 std::function<void(cudaStream_t)> function_mapped_p
207     std::bind(launch_float_addition_mapped_pinned_me
208         m_input_1, m_input_2, n, std::placeho:
209 float const latency_non_mapped_pinned_memory{measure
210     function_non_mapped_pinned_memory, stream, num_
211 float const latency_mapped_pinned_memory{measure_pe
212     function_mapped_pinned_memory, stream, num_repe
213 std::cout << std::fixed << std::setprecision(3)
214     << "CUDA Kernel With Non-Mapped Pinned Mer
215     << latency_non_mapped_pinned_memory << " r
216 std::cout << std::fixed << std::setprecision(3)
217     << "CUDA Kernel With Mapped Pinned Memory
218     << latency_mapped_pinned_memory << " ms" <
219
220 CHECK_CUDA_ERROR(cudaFree(d_input_1));
221 CHECK_CUDA_ERROR(cudaFree(d_input_2));
222 CHECK_CUDA_ERROR(cudaFree(d_output));
223 CHECK_CUDA_ERROR(cudaFreeHost(h_input_1));
224 CHECK_CUDA_ERROR(cudaFreeHost(h_input_2));
225 CHECK_CUDA_ERROR(cudaFreeHost(h_output));
226 CHECK_CUDA_ERROR(cudaFreeHost(a_input_1));
227 CHECK_CUDA_ERROR(cudaFreeHost(a_input_2));
228 CHECK_CUDA_ERROR(cudaFreeHost(a_output));
229 CHECK_CUDA_ERROR(cudaStreamDestroy(stream));
230 }

```

Discrete GPU

This is the latency profiling on a desktop that has Intel Core i9-9900K CPU and NVIDIA RTX 3090 GPU.



```
1 $ nvcc mapped_memory.cu -o mapped_memory -std=c++14
2 $ ./mapped_memory
3 CUDA Kernel With Non-Mapped Pinned Memory Latency: 0.964 ms
4 CUDA Kernel With Mapped Pinned Memory Latency: 0.631 ms
```

We could see that for memory-bound kernel, on a platform that uses discrete GPU, separate host memory, and device memory, using mapped pinned memory is almost 30% faster than using non-mapped pinned memory.

Integrated GPU

This is the latency profiling on an NVIDIA Jetson Xavier.



```
1 $ nvcc mapped_memory.cu -o mapped_memory -std=c++14
2 $ ./mapped_memory
3 CUDA Kernel With Non-Mapped Pinned Memory Latency: 2.343 ms
4 CUDA Kernel With Mapped Pinned Memory Latency: 0.431 ms
```

We could see that for memory-bound kernel, on a platform that uses integrated GPU and unified memory, using mapped pinned memory is almost 6x faster than using non-mapped pinned memory. This is because the using mapped memory truly eliminated the memory copy between host and device on unified memory.

Caveats

CUDA zero copy memory disables data cache on GPUs, there might be performance drop for math bound kernels.

References

- [Function Binding and Performance Measurement](#)
- [NVIDIA CUDA Memory Management](#)
- [Zero Copy Memory - CUDA Best Practice Guide](#)

CUDA Zero Copy Mapped Memory

<https://leimao.github.io/blog/CUDA-Zero-Copy-Mapped-Memory/>

Author	Posted on	Updated on	Licensed under
Lei Mao	12-16-2022	12-16-2022	



330
Shares

LIKE THIS ARTICLE? SUPPORT THE AUTHOR WITH



Paypal



Buy me a coffee

◀ Coyote Valley Open Space Preserve 徒步

虚假的快乐 ▶

Comments

8 Comments

 Login ▾

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS  2

Share

Best Newest Oldest**Jang**

a year ago

Integrated gpu shares memory between gpu and cpu, so if memory is allocated using cudaMallocHost(or cudaHostAlloc with cudaHostAllocMapped), both cpu and gpu can access the memory area without memcpy. And I was understanding this as zerocopy. By the way, I still don't understand how mapped and non-mapped are different in zerocopy. Could you please explain this in a little more detail?

  Reply Share >**Lei Mao** Mod Jang

a year ago edited

Memory allocated with `cudaMallocHost` is page-locked but not mapped. As is shown in the code, you will still need to do memory copy if you want to use the data on the memory on GPU.

Memory allocated with `cudaHostAlloc` with the `cudaHostAllocMapped` flag is page-locked and mapped. The GPU can directly use the data on the memory without memory copy, achieving zero-copy.

  Reply Share >**Jang** Lei Mao

a year ago

Thank you for your kind response. But as far as I know, the memory allocated by using the cudaMallocHost function can be zerocopied without memory copying between host and device.

Even if the launch_float_addition_non_mapped_pinned_memory function in the example code is changed as below, verify_host_memory will pass.

```
void
launch_float_addition_non_mapped_pinned_memory
(
```



```
float* h_output, float const* h_input_1, float
const* h_input_2,
float* d_output, float* d_input_1, float*
d_input_2, uint32_t n,
cudaStream_t stream)
{
/*CHECK_CUDA_ERROR(cudaMemcpyAsync(d_input_1,
h_input_1, n * sizeof(float),
cudaMemcpyHostToDevice, stream));
CHECK_CUDA_ERROR(cudaMemcpyAsync(d_input_2,
h_input_2, n * sizeof(float),
cudaMemcpyHostToDevice, stream));*/
dim3 const threads_per_block{1024};
dim3 const blocks_per_grid{32};
float_addition<<<blocks_per_grid,
threads_per_block,="" 0,="" stream="">>>(
h_output, h_input_1, h_input_2, n);
//d_output, d_input_1, d_input_2, n);
CHECK_LAST_CUDA_ERROR();
/*CHECK_CUDA_ERROR(cudaMemcpyAsync(h_output,
d_output, n * sizeof(float),
cudaMemcpyDeviceToHost, stream));*/
}
```

In my practice, I am using `cudaMallocHost` function for zerocopy, am I using it wrong?

o o Reply Share ›



Lei Mao Mod

→ Jang

a year ago edited

Thank you very much. I double checked the CUDA Runtime API documentation on [Unified Addressing](#). It seems that my understanding of `cudaMallocHost`` was slightly erroneous and your usage was correct. I will update this article once I get a chance in a few weeks. Thanks a lot for your

